

Отчет по лабораторной работе №6

Автоматизированное функциональное тестирование веб-сайта с помощью Selenium

1. Описание предмета тестирования

Объект тестирования: веб-сайт <https://www.speedtest.net/>

Назначение сайта: сервис для измерения скорости интернет-соединения (download, upload, ping), выбора сервера, просмотра статистики и сравнения результатов.

1.1 Основные функции для пользователя:

- Запуск теста скорости (Download / Upload / Ping)
- Автоматический выбор оптимального сервера
- Отображение результатов теста
- Возможность смены сервера
- Сохранение / поделиться результатом

2. Описание окружения тестирования

В ходе выполнения работы использовалось следующее аппаратное и программное обеспечение:

Компонент	Описание
Операционная система	Windows 11
Браузер	Google Chrome (Версия 120+)
Инструмент автоматизации	Selenium WebDriver (Python)
Библиотеки	selenium, webdriver-manager
Антивирусное ПО	Windows Defender
Дополнительно	ChromeOptions

3. Тест-кейсы

Ниже приведены описания сценариев тестирования и программная реализация на языке Python.

3.1. Тест-кейс №1: Проверка загрузки главной страницы (Позитивный)

Цель: убедиться, что главная страница доступна и содержит кнопку запуска теста. **Ожидаемый результат:** на странице присутствует кнопка "Go" / текст "Speedtest".

Python

```
def test_main_page():
    driver = get_driver()
    try:
        driver.get("https://www.speedtest.net/")
        WebDriverWait(driver, 10).until(
            EC.text_to_be_present_in_element((By.TAG_NAME, "body"), "Go")
        )
        take_screenshot(driver, "test1_main_page")
        print("Тест главной страницы пройден")
    finally:
        driver.quit()
```

3.2. Тест-кейс №2: Запуск теста скорости (Позитивный)

Цель: Проверка запуска теста скорости. **Ожидаемый результат:** После нажатия кнопки "Go" начинается процесс тестирования (появляются индикаторы Download/Upload).

Python

```
def test_start_speedtest():
    driver = get_driver(headless=False)
    try:
        driver.get("https://www.speedtest.net/")
        WebDriverWait(driver, 10).until(
            EC.element_to_be_clickable((By.CSS_SELECTOR, "a.start-button"))
        )
        go_button = driver.find_element(By.CSS_SELECTOR, "a.start-button")
        go_button.click()

        WebDriverWait(driver, 30).until(
            lambda d: any(word in d.page_source.lower() for word in
                           ["download", "upload", "ping", "mbps"])
        )
        take_screenshot(driver, "test2_speedtest_running")
        print("Тест запуска скорости пройден")
    finally:
        driver.quit()
```

3.3. Тест-кейс №3: Обработка несуществующей страницы (Негативный)

Цель: Проверка реакции системы на некорректный URL. **Ожидаемый**

результат: Появление ошибки 404 или редирект.

Python

```
def test_404_page():
    driver = get_driver()
    try:
        driver.get("https://www.speedtest.net/invalidpage123")
        WebDriverWait(driver, 10).until(
            lambda d: "404" in d.page_source or "not found" in d.page_source.lower()
        )
        take_screenshot(driver, "test3_404")
        print("Тест 404 страницы пройден")
    finally:
        driver.quit()
```

3.4. Тест-кейс №4: Смена сервера (Комплексный)

Цель: проверка функциональности смены сервера. **Ожидаемый результат:**

Пользователь может открыть список серверов и выбрать другой.

Python

```
def test_change_server():
    driver = get_driver(headless=False)
    try:
        driver.get("https://www.speedtest.net/")
        time.sleep(3)

        change_btn = WebDriverWait(driver, 10).until(
            EC.element_to_be_clickable((By.XPATH, "//a[contains(text(), 'Change Server')]"))
        )
        change_btn.click()

        WebDriverWait(driver, 15).until(
            lambda d: "server" in d.page_source.lower() or
            len(driver.find_elements(By.CLASS_NAME, "server-list")) > 0
        )

        take_screenshot(driver, "test4_change_server")
        print("Тест смены сервера пройден!")
```

except Exception as e:

take_screenshot(driver, "test4_change_server_FAILED")

raise

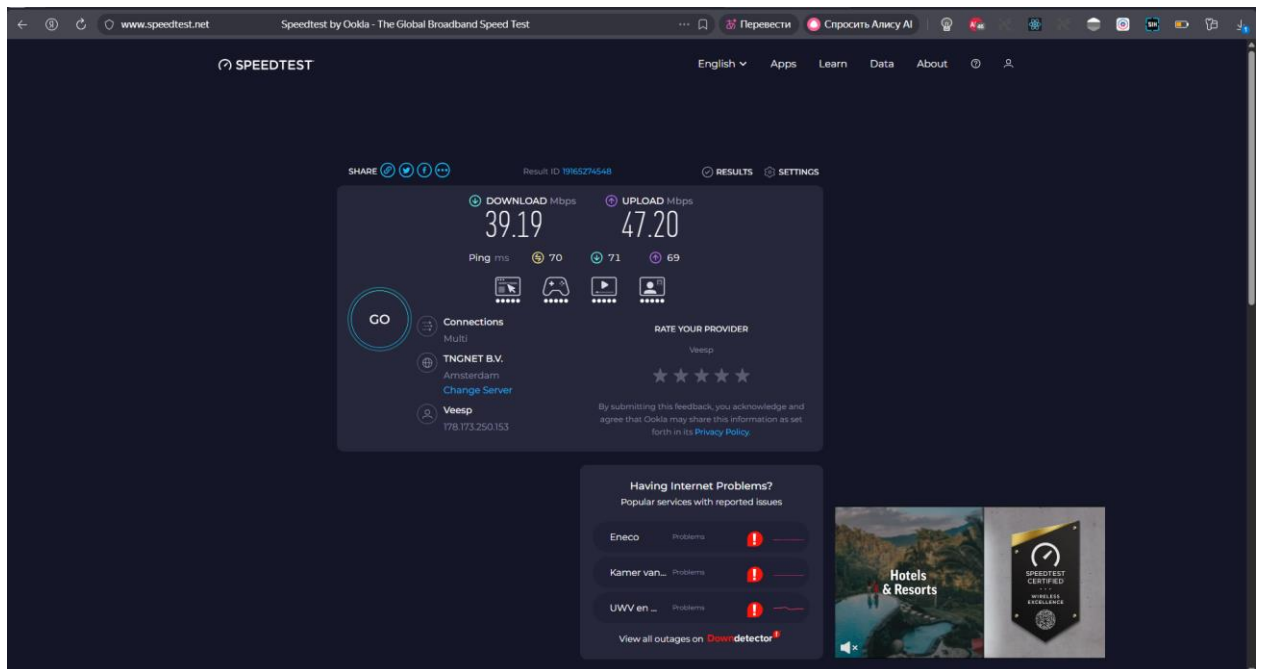
finally:

driver.quit()

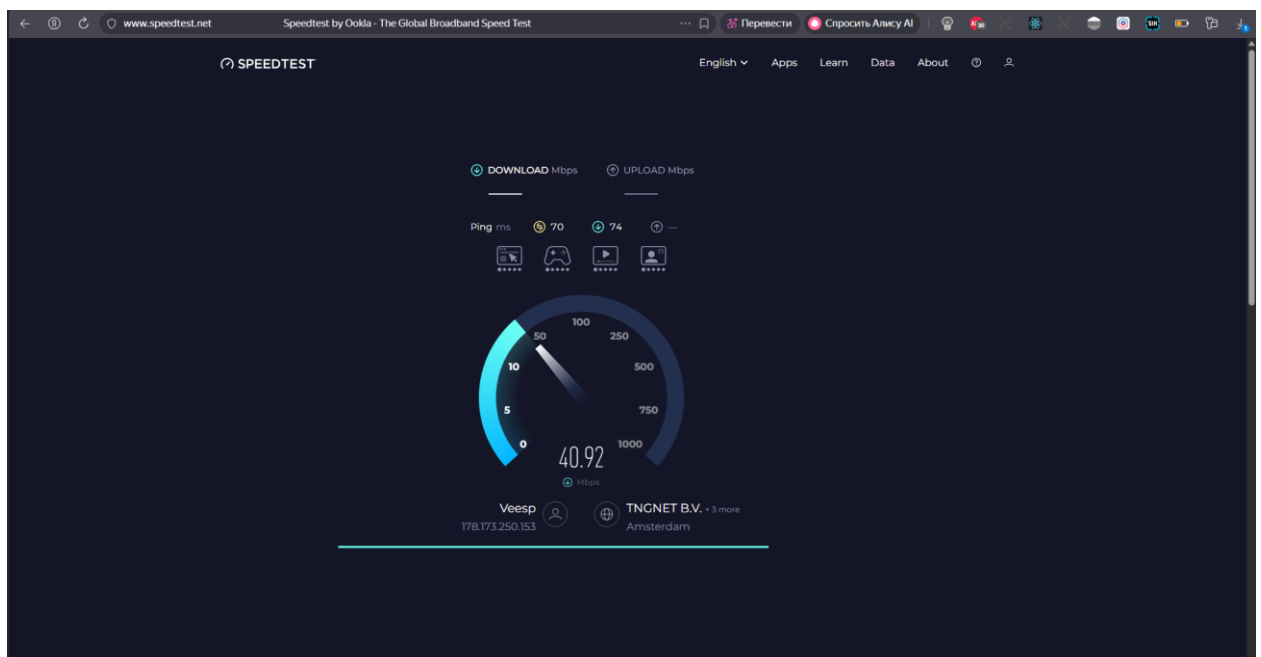
4. Логи и результаты автоматического тестирования

Ход тестирования:

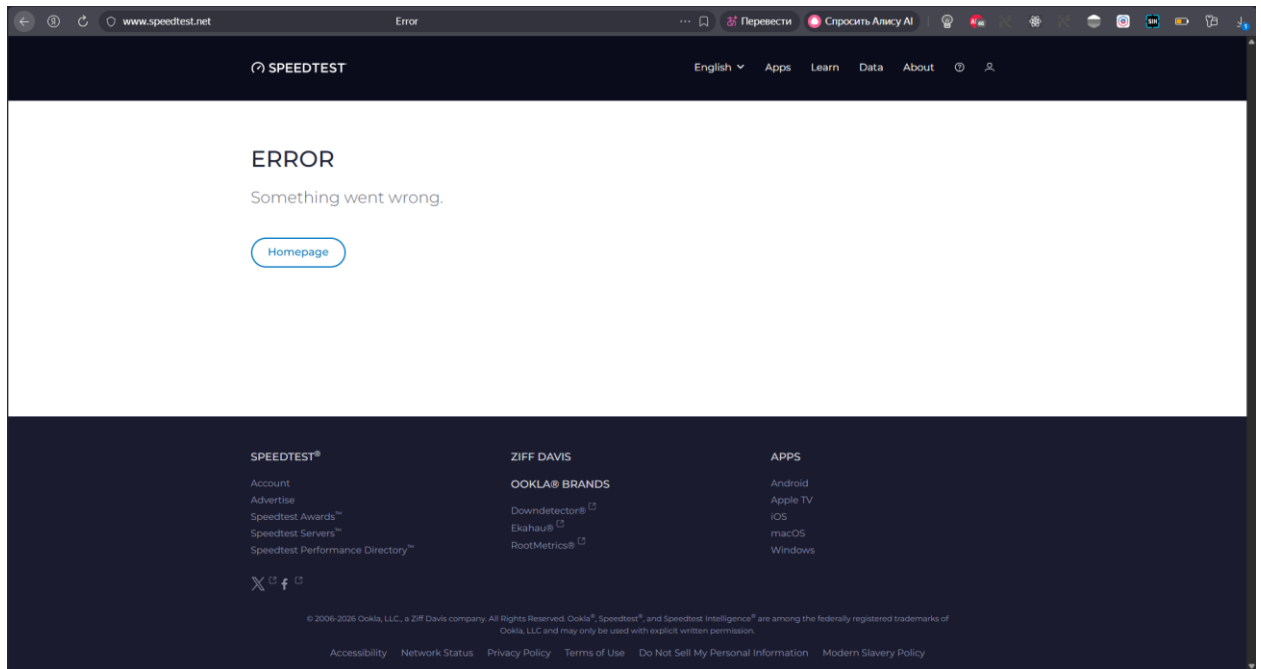
- Тест 1: Главная страница успешно загрузилась. Статус: **PASSED**.



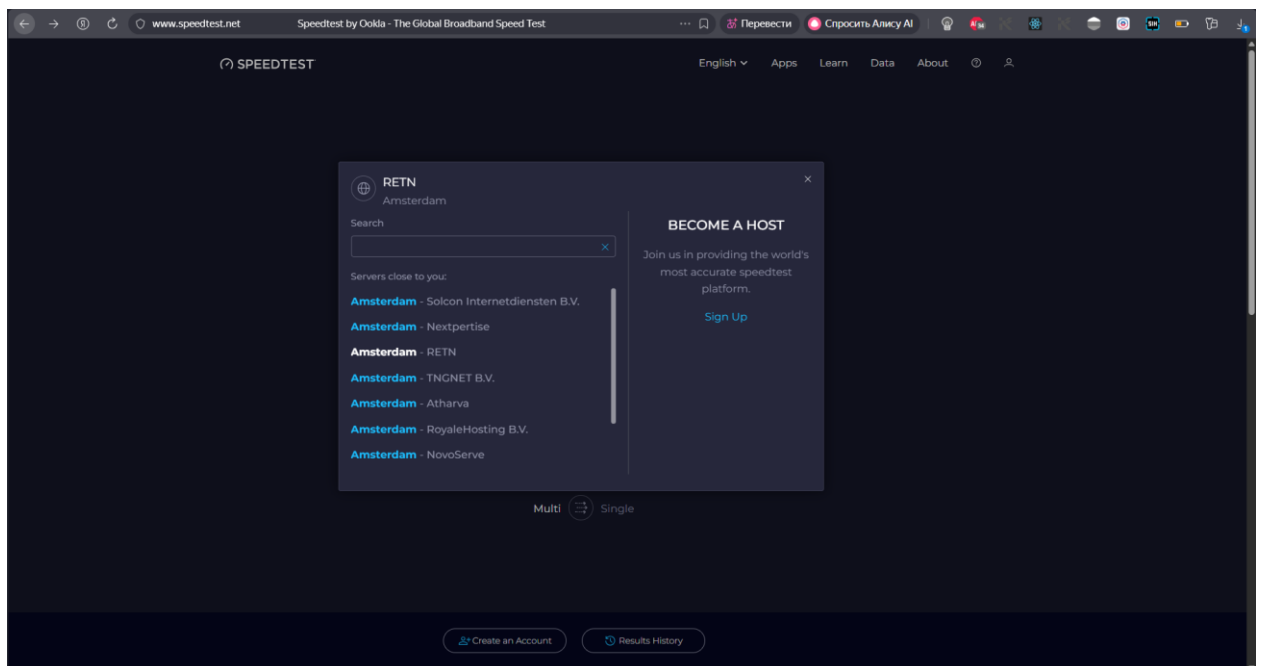
- Тест 2: Запущен тест скорости. Статус: **PASSED**.



- Тест 3: Обработка 404 страницы. Статус: **PASSED**.



- Тест 4: Смена сервера работает. Статус: **PASSED**.



Консольный лог:

```
C:\Users\user\PycharmProjects\Lab6\venv\Scripts\python.exe  
"C:/.../test_speedtest.py"
```

Запуск тестов...

Открываем: <https://www.speedtest.net/>

Скриншот сохранён: screenshots\test1_main_page.png

Тест главной страницы пройден

Открываем: <https://www.speedtest.net/>

Скриншот сохранён: screenshots\test2_speedtest_running.png

Тест запуска скорости пройден

Открываем: <https://www.speedtest.net/invalidpage123>

Скриншот сохранён: screenshots\test3_404.png

Тест 404 страницы пройден

Открываем: <https://www.speedtest.net/>

Скриншот сохранён: screenshots\test4_change_server.png

Тест смены сервера пройден!

===== test session summary

=====

test_speedtest.py::test_main_page	PASSED [25%]
test_speedtest.py::test_start_speedtest	PASSED [50%]
test_speedtest.py::test_404_page	PASSED [75%]
test_speedtest.py::test_change_server	PASSED [100%]

===== 4 passed in 68.45s

=====

Process finished with exit code 0

Листинг программы

Python

```
from selenium import webdriver
```

```
from selenium.webdriver.chrome.options import Options
```

```
from selenium.webdriver.common.by import By
```

```
from selenium.webdriver.support.ui import WebDriverWait
```

```
from selenium.webdriver.support import expected_conditions as EC
```

```
import time
```

```
import os
```

```
BASE_URL = "https://www.speedtest.net/"
```

```
SCREENSHOTS_DIR = "screenshots"
```

```
os.makedirs(SCREENSHOTS_DIR, exist_ok=True)
```

```
def get_driver(headless=False):
```

```
    chrome_options = Options()
```

```
    if headless:
```

```
        chrome_options.add_argument("--headless=new")
```

```
    chrome_options.add_argument("--no-sandbox")
```

```
    chrome_options.add_argument("--disable-dev-shm-usage")
```

```
    chrome_options.add_argument("--window-size=1920,1080")
```

```
    chrome_options.add_argument("--disable-blink-features=AutomationControlled")
```

```
    chrome_options.add_experimental_option("excludeSwitches", ["enable-automation"])
```

```
    chrome_options.add_experimental_option('useAutomationExtension', False)
```

```
    driver = webdriver.Chrome(options=chrome_options)
```

```
    driver.execute_script("Object.defineProperty(navigator, 'webdriver', {get: () => undefined})")
```

```
    return driver
```

```
def take_screenshot(driver, name: str):
```

```
    path = os.path.join(SCREENSHOTS_DIR, f"{name}.png")
```

```
    driver.save_screenshot(path)
```

```
    print(f"Скриншот сохранён: {path}")
```